01-js-fundamentals

01. JavaScript Fundamentals

> **Level:** 🟢 Beginner
> **Jam:** 8 (4 minggu × 2 sesi)
> **Prasyarat:** —
> **Output:** Kumpulan script JS, utility functions, CLI apps

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Nulis kode JavaScript sendiri dari nol
- Paham variable, tipe data, kondisi, loop
- Pake array & object buat nyimpen data
- Bikin function sendiri
- Handling error + async/await + fetch API

Materi

Sesi	Topik	File
1	Programming mindset + variable & tipe data	01-variables-types.md
2	Control flow (if/else, switch, loop)	02-control-flow.md
3	Array & Object	03-arrays-objects.md
4	Functions	04-functions.md
5	Async, Error Handling & Fetch	05-async-errors.md

Output Akhir Modul

> **CLI Utility App** — program node.js interaktif (todo list / catatan / password generator)

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- ”Explain this code line by line”
- ”Find the bug in this code”
- ”Refactor this to use arrow functions”
- ”Generate 3 test cases for this function”

1.1 Variable & Tipe Data

Variable

```
// let — bisa diubah
let nama = "Budi";
nama = "Andi"; // OK

// const — tidak bisa diubah
const umur = 17;
umur = 18; // ERROR!

// var — JANGAN PAKAI (old)
```

Tipe Data

Tipe	Contoh	Keterangan
string	"Halo", 'Halo', \Halo“	Teks
number	42, 3.14, -5	Angka
boolean	true, false	Benar/salah
undefined	let x;	Belum diisi
null	let x = null;	Sengaja kosong
object	{nama: "Budi"}	Kumpulan data

```
const nama = "Budi";
const umur = 17;
const isStudent = true;

console.log(typeof nama); // `string`
console.log(typeof umur); // `number`
```

Template Literals

```
const nama = "Budi";
const nilai = 85;
```

```javascript
// Old way
console.log("Nama: " + nama + ", Nilai: " + nilai);

// Modern (template literal)
console.log(`Nama: ${nama}, Nilai: ${nilai}`);
```

Operator

```javascript
// Aritmatika
let a = 10 + 5;  // 15
let b = 10 - 5;  // 5
let c = 10 * 5;  // 50
let d = 10 / 5;  // 2
let e = 10 % 3;  // 1 (modulus / sisa bagi)

// Perbandingan
console.log(10 > 5);  // true
console.log(10 === 10); // true (=== strict equal)
console.log(10 == "10"); // true (loose, jangan dipake)
console.log(10 === "10"); // false

// Logical
console.log(true && false); // false (AND)
console.log(true || false); // true (OR)
console.log(!true); // false (NOT)
```

Latihan

1. Bikin variable nama, umur, hobi. Print pakai template literal
2. Program konversi suhu: Celsius -> Fahrenheit
3. Program hitung luas lingkaran (input jari-jari)
4. Cek apakah angka genap/ganjil pake modulus

1.2 Control Flow

If/Else

```javascript
const nilai = 85;

if (nilai >= 90) {
  console.log("Grade A");
} else if (nilai >= 75) {
  console.log("Grade B");
```

```
} else if (nilai >= 60) {
  console.log("Grade C");
} else {
  console.log("Grade D - Remedial");
}
```

Ternary Operator (if/else pendek)

```
const umur = 17;
const status = umur >= 17 ? "Bisa bikin SIM" : "Belum bisa";
```

Switch

```
const hari = "senin";

switch (hari) {
  case "senin":
    console.log("Mulai ngoding");
    break;
  case "jumat":
    console.log("Alhamdulillah");
    break;
  default:
    console.log("Ngoding aja");
}
```

Loops

```
// For loop
for (let i = 1; i <= 5; i++) {
  console.log(`Iterasi ke-${i}`);
}

// While loop
let tebakan = 0;
const jawaban = 7;
while (tebakan !== jawaban) {
  tebakan = Math.floor(Math.random() * 10);
  console.log(`Tebakan: ${tebakan}`);
}

// Array loop
const fruits = ["Apel", "Mangga", "Jeruk"];
for (const fruit of fruits) {
```

```
  console.log(fruit);
}
```

Latihan

1. Program kalkulator sederhana (+, -, *, /)
2. Program tebak angka (komputer random 1-10, user nebak)
3. Program tabel perkalian (input 5 -> output 5x1 sampai 5x10)
4. Program cetak pola bintang

```
*
**
***
****
*****
```

1.3 Array & Object

Array

```javascript
// Bikin array
const fruits = ["Apel", "Mangga", "Jeruk", "Pisang"];

// Akses
console.log(fruits[0]); // "Apel"
console.log(fruits.length); // 4

// Manipulasi
fruits.push("Anggur");   // Tambah di akhir
fruits.pop();            // Hapus dari akhir
```

Array Methods (Modern, Penting!)

```javascript
const numbers = [1, 2, 3, 4, 5];

// map - transformasi
const doubled = numbers.map(n => n * 2);
// [2, 4, 6, 8, 10]

// filter - saring
const even = numbers.filter(n => n % 2 === 0);
// [2, 4]

// reduce - akumulasi
```

```
const sum = numbers.reduce((acc, n) => acc + n, 0);
// 15
```

Object

```
const siswa = {
  nama: "Budi",
  umur: 17,
  kelas: "XII RPL",
  nilai: {
    matematika: 85,
    bahasa: 90,
  },
};

// Akses
console.log(siswa.nama);  // "Budi"
console.log(siswa.nilai.matematika); // 85

// Destructuring
const { nama, umur } = siswa;
console.log(nama, umur);

// Spread operator
const siswaBaru = { ...siswa, nama: "Andi" };
```

Latihan

1. Array: buat daftar belanja, tambah item, hapus item, tampilkan
2. Array: filter harga > 10000 dari array produk
3. Object: buat data 5 siswa, hitung rata-rata nilai
4. Object + Array: contact book (tambah, cari, hapus kontak)

1.4 Functions

Arrow Function (Modern)

```
// Satu parameter, satu baris
const sapa = nama => `Halo, ${nama}!`;

// Multiple params
const tambah = (a, b) => a + b;

// Multiple baris
```

```javascript
const hitung = (a, b, operasi) => {
  switch (operasi) {
    case "+": return a + b;
    case "-": return a - b;
    case "*": return a * b;
    case "/": return b !== 0 ? a / b : "Error: bagi 0";
    default: return "Error";
  }
};
```

Default Parameter

```javascript
const sapa = (nama = "Teman") => `Halo, ${nama}!`;
console.log(sapa());        // "Halo, Teman!"
console.log(sapa("Budi")); // "Halo, Budi!"
```

Higher-Order Functions (callbacks)

```javascript
function prosesAngka(angka, callback) {
  return callback(angka);
}

const double = n => n * 2;
console.log(prosesAngka(5, double)); // 10

// Langsung:
console.log(prosesAngka(5, n => n * 2)); // 10
```

Latihan

1. Bikin function: `generatePassword(length)` - return random password
2. Bikin function: `isPalindrome(str)` - cek palindrome
3. Bikin function: `validateEmail(email)` - validasi email
4. Bikin utility library dengan 10+ function

1.5 Async, Error Handling & Fetch

Error Handling

```javascript
function bagi(a, b) {
  try {
    if (b === 0) throw new Error("Ga bisa bagi dengan 0");
    return a / b;
```

```
  } catch (error) {
    console.error(error.message);
    return null;
  }
}
```

Async/Await

```
function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

async function main() {
  console.log("Mulai...");
  await delay(2000);
  console.log("2 detik kemudian...");
}
```

Fetch API

```
async function getJoke() {
  try {
    const response = await fetch("https://official-joke-api.appspot.com/random_j
    const data = await response.json();
    console.log(`${data.setup}`);
    console.log(`${data.punchline}`);
  } catch (error) {
    console.error("Gagal ambil data:", error.message);
  }
}
getJoke();
```

Debug Challenge

Cari & fix bug:

```
// Bug 1: infinite loop
function countDown(n) {
  while (n > 0) {
    console.log(n);
  }
}

// Bug 2: NaN
function average(arr) {
```

```
  let total = 0;
  for (let i = 0; i <= arr.length; i++) {
    total += arr[i];
  }
  return total / arr.length;
}

// Bug 3: undefined name
const user = { name: "Budi", greet: () => console.log(`Halo ${this.name}`) };
user.greet();
```

Latihan Akhir

1. Fetch dari public API (cuaca / berita / crypto) - tampilkan rapi
2. Handle error kalo API down atau network error
3. Gabungin semua konsep: function + array + object + async + error handling